42

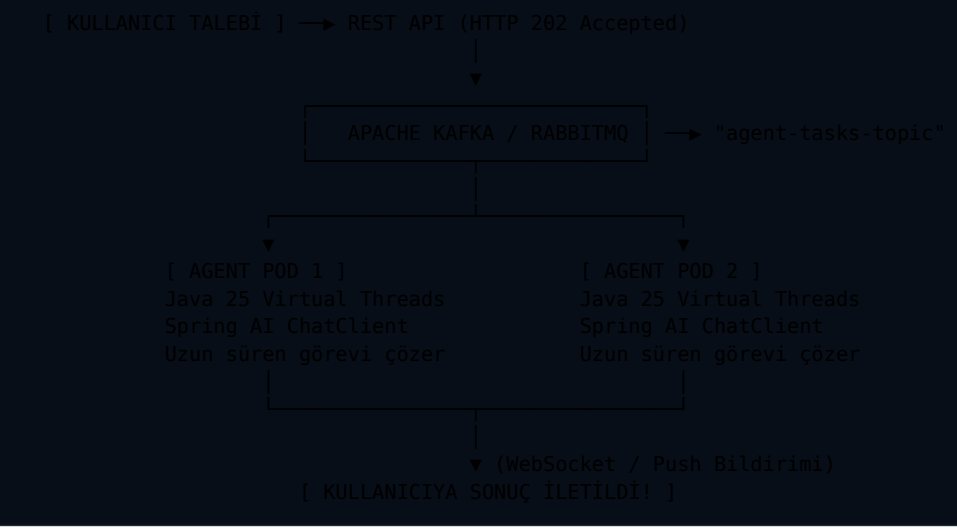
BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

Spring AI ile Dağıtık Asenkron Ajan Orkestrasyonu

Distributed Asynchronous Agent Orchestration with Spring AI & Event Streams

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda sipariş verdiğinizde garson yemeğiniz pişene kadar 30 dakika boyunca masanızın başında dikilip beklemez! Masanıza bir numara bırakır (HTTP 202 Accepted), sipariş fişini mutfağa asar (Kafka Mesajı) ve diğer masalara hizmet etmeye gider. Mutfaktaki aşçılar (Arka plan Ajanları) yemeği hazırlar ve bittiğinde zil çalar; garson yemeği masanıza getirir! Dağıtık Ajan Orkestrasyonu tam olarak budur: Saatler sürebilecek büyük yapay zeka işlerini kullanıcının ekranını kilitlemeden arka planda asenkron olarak çözmektir!

🇬🇧 ENGLISH (ELI5)

When ordering a complex banquet at a restaurant, the waiter doesn't stand frozen at your table for 45 minutes while the roast cooks. They hand you an order buzzer (HTTP 202 Accepted), pin the ticket to the kitchen queue (Kafka Event Stream), and immediately serve other patrons. Kitchen chefs (Distributed AI Agent Pods) process the orders asynchronously. When ready, the buzzer flashes! Distributed Agent Orchestration decouples heavy AI tasks from synchronous web requests!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık Ajan Mimarisi: 1) Asenkron Kabul (HTTP 202 Accepted): İstemciye hemen bir `taskId` dönülür, HTTP bağlantısı kapatılır. 2) Message Broker (Kafka / RabbitMQ): Ajan görevleri bir kuyruğa bırakılır; kuyruk arkasında çalışan onlarca Spring Boot pod'u görevleri paralel tüketir. 3) Virtual Threads ile Paralel Ajanlar: Java 25 sayesinde her bir pod içinde binlerce sanal iş parçacığı aynı anda farklı ajan döngülerini bloklanmadan yönetir. 4) Sonuç Bildirimi: Görev bittiğinde WebSocket, Webhook veya Mobil Push bildirimi ile istemci bilgilendirilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Distributed Agent Architecture & Asynchronous Topologies: 1) Decoupled Ingress: The API gateway accepts the request, returns `HTTP 202 Accepted` with a `tracking_id`, and dispatches an event. 2) Event-Driven Task Scheduling: Kafka or RabbitMQ buffers tasks, distributing workloads across scalable Spring Boot worker pools. 3) High Concurrency via Project Loom: Java 25 Virtual Threads allow a single worker pod to host thousands of concurrent, I/O-blocked agent execution loops without thread pool starvation. 4) Push Notification Delivery: Real-time status updates broadcast via WebSockets or Webhooks upon completion.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25 & Spring AI: Asenkron Görev Başlatıcı ve Virtual Thread Modeli
import java.util.UUID;
import java.util.concurrent.CompletableFuture;

public class DistributedAgentDispatcher {
    public record AgentTaskTicket(String taskId, String status) {}

    public static AgentTaskTicket submitAgentTask(String userGoal) {
        String taskId = UUID.randomUUID().toString();

        // Java 25 Virtual Threads ile arka planda asenkron ajan çalıştırma
        Thread.ofVirtual().start(() -> {
            IO.println("Arka Planda Ajan Çalışıyor (Task: " + taskId + "): " + userGoal);
            try {
                Thread.sleep(1500); // LLM ve Tool çağrıları simülasyonu
                IO.println("Ajan Görevi Başarıyla Tamamladı! (Task: " + taskId + ")");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // İstemciye anında bilet dönülür (HTTP 202)
        return new AgentTaskTicket(taskId, "ACCEPTED");
    }
}

void main() {
    var ticket = DistributedAgentDispatcher.submitAgentTask("Tüm müşteri şikayetlerini analiz et");
    IO.println("İstemciye Dönen Yanıt: Task ID = " + ticket.taskId() + " | Durum = " +
ticket.status());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mimari Kural: Uzun süren (10 saniyeden fazla) hiçbir ajan işini senkron bir HTTP GET/POST isteği içinde tutmayın! Bağlantı zaman aşımına (Timeout - 504 Gateway Timeout) uğrar. İşi daima asenkron kuyruğa atıp istemciye taskId dönün.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Gateway timeout antipattern: Never execute multi-step agent loops inside a synchronous HTTP request lifecycle. Ingress proxies terminate connections after 30-60 seconds. Always decouple execution via asynchronous queues and WebSockets.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Uzun süren ajan görevleri HTTP 202 Accepted ve asenkron mesaj kuyruklarıyla yönetilmelidir.
- Java 25 Virtual Threads, binlerce eşzamanlı ajan döngüsünü minimum bellek ile işletir.
- Tamamlanan görevler WebSocket veya Webhook bildirimleriyle istemciye ulaştırılır.

🎯 Key Takeaways for the Interview

- Long-running agent workflows mandate asynchronous HTTP 202 patterns and message queues.
- Java 25 Virtual Threads sustain thousands of concurrent agent loops with near-zero overhead.
- Completed task payloads are streamed back to users via WebSockets or webhook callbacks.